CRUD Bagian 1 — Create & Read

Pemrograman Web • 9 slides • Generated on 11 October 2026, 22:42

CRUD Bagian 1: Create & Read

- **Materi:**
  - INSERT dengan `ExecContext` dan `LastInsertId`
  - Query berparameter, anti SQL injection
  - `QueryRowContext` dan `sql.ErrNoRows`
  - Hasil query ke `struct`
  - Context dari request

Go Time Format: Urutan 1–7

Slide 2

Reference time: `Mon Jan 2 15:04:05 MST 2006`

Urutan	Komponen	Layout	Contoh output
1	Bulan	`1` · `01` · `Jan` · `January`	10 · 10 · Oct · October
2	Tanggal	`2` · `02` · `_2`	7 · 07 · " 7"
3	Jam	`3` · `03` · `15`	6 · 06 · 18
4	Menit	`4` · `04`	5 · 05
5	Detik	`5` · `05`	9 · 09
6	Tahun	`06` · `2006`	26 · 2026
7	Zona waktu	`-0700` · `-07:00` · `MST`	+0700 · +07:00 · WIB

Tambahan: hari `Mon` / `Monday`, AM/PM `PM`, milidetik `.000`

Route Daftar Produk

Slide 3

Method	Path	Fungsi
GET	`/produk`	daftar produk
GET	`/produk/baru`	form produk
POST	`/produk`	simpan produk baru
GET	`/produk/{id}`	detail satu produk

CREATE: ExecContext

Slide 4

- Untuk INSERT, UPDATE, DELETE pakai `ExecContext`, hasilnya `sql.Result`

```
res, err := db.ExecContext(ctx,
    "INSERT INTO produk (nama, harga, stok, keterangan, created_at) VALUES (?, ?, ?, ?, ?)",
   nama, harga, stok, keterangan, created_at
)
if err != nil {
    return 0, err
}
id, err := res.LastInsertId()
```

- `?` = **placeholder**, nilai dikirim terpisah dari teks SQL
- `LastInsertId()` = `id` dari `AUTO_INCREMENT` baris yang baru masuk
    - Berguna untuk redirect ke halaman detail: `/produk/7`

```
http.Redirect(w, r, fmt.Sprintf("/produk/%d", id), http.StatusSeeOther)
```

Query Berparameter

Slide 5

- Menyusun SQL dengan string = **SQL injection**

```
// tidak disarankan
q := "SELECT id,nama FROM produk WHERE id = '" + id + "'"
q := fmt.Sprintf("SELECT id, nama FROM produk WHERE id = '%s'", id)
```

- Gunakan placeholder:

```
db.QueryContext(ctx, "SELECT id, nama FROM produk WHERE id = ?", id)
```

- Dengan placeholder, input **selalu** dianggap data, tidak akan menjalankan perintah SQL
- Contoh untuk `LIKE`:

```
db.QueryContext(ctx,
    "SELECT id, nama FROM produk WHERE nama LIKE ?", "%"+kata+"%")
```

- Placeholder hanya untuk **nilai**, bukan nama tabel/kolom

SQL Injection

Slide 6

- Kode rentan, pencarian data:

```
q := "SELECT id, judul FROM catatan WHERE judul LIKE '%" + kata + "%'"
rows, err := db.QueryContext(ctx, q)
```

- User mengetik:

```
' UNION SELECT id, password FROM users --
```

- SQL yang dijalankan:

```
SELECT id, judul FROM catatan WHERE judul LIKE '%'
UNION SELECT id, password FROM users -- %'
```

- Isi tabel lain ikut tampil di halaman pencarian

READ: Satu Baris

Slide 7

```
func ambilProduk(ctx context.Context, id int64) (Produk, error) {
    var p Produk
    err := db.QueryRowContext(ctx,
        "SELECT id, nama, harga, stok FROM prodiuk WHERE id = ?", id,
    ).Scan(&p.ID, &p.Name, &p.Price, &p.Stok)
    return p, err
}
```

- `QueryRowContext` untuk hasil **paling banyak satu** baris, tidak perlu `rows.Close()`
- Tidak ada baris → `Scan` mengembalikan `sql.ErrNoRows`
- Di handler, bedakan "tidak ada" dengan "error":

```
p, err := ambilProduk(r.Context(), id)
if errors.Is(err, sql.ErrNoRows) {
    http.NotFound(w, r)       // 404
    return
}
if err != nil {
```

```
    serverError(w, err)       // 500
    return
}
```

Context dari Request

Slide 8

- Setiap request punya context: `r.Context()`
- Context ini **dibatalkan** saat user menutup tab atau koneksi putus

```
func daftarHandler(w http.ResponseWriter, r *http.Request) {
    produk, err := ambilDaftarProduk(r.Context(), 50)
    ...
}
```

- Oper `r.Context()` ke semua fungsi yang mengakses database
    - Ketika user membatalkan, koneksi cepat kembali ke pool
- Konvensi Go: `ctx context.Context` selalu jadi **parameter pertama**

```
func ambilProduk(ctx context.Context, id int64) (Produk, error)
```

Error Database: Log vs User

Slide 9

- Error dari MySQL bisa berisi nama tabel, kolom, bahkan potongan query

```
Error 1054 (42S22): Unknown column 'judl' in 'field list'
```

- Informasi ini berguna untuk **developer**, berbahaya kalau dilihat **penyerang**

- Satu helper untuk tipe error tertentu:

```
// error 500
func serverError(w http.ResponseWriter, err error) {
    log.Printf("error: %v", err)
    http.Error(w, "terjadi kesalahan di server", http.StatusInternalServerError)
}

// error 400
func clientError(w http.ResponseWriter, status int, pesan string) {
    http.Error(w, pesan, status)
}

// error 404
```

```
func notFound(w http.ResponseWriter) {
    clientError(w, http.StatusNotFound, "data tidak ditemukan")
}
```